

SeguraMente

Documentação técnica do MVP funcional — Desafio 5 InsurMinds

Versão: 3.0

Data: 18 de agosto de 2026

Status: pacote de entrega validado localmente

1. Resumo executivo

O SeguraMente é um MVP de comunicação proativa com segurados baseado na análise de eventos meteorológicos. A solução consulta a Open-Meteo, normaliza os dados meteorológicos, classifica eventos relevantes por limiares explícitos, cruza o resultado com uma base sintética de perfis, aplica regras de negócio, gera mensagens preventivas por meio de um modelo de linguagem, apresenta o conteúdo para revisão humana e registra o envio como simulação.

A implementação atende às etapas mínimas do Desafio 5: coleta em fonte externa, identificação de eventos, regras de decisão, geração automática e simulação de notificações [1]. O pacote inclui código-fonte, interface Streamlit, testes automatizados, README, licença MIT e evidências JSON de execução.

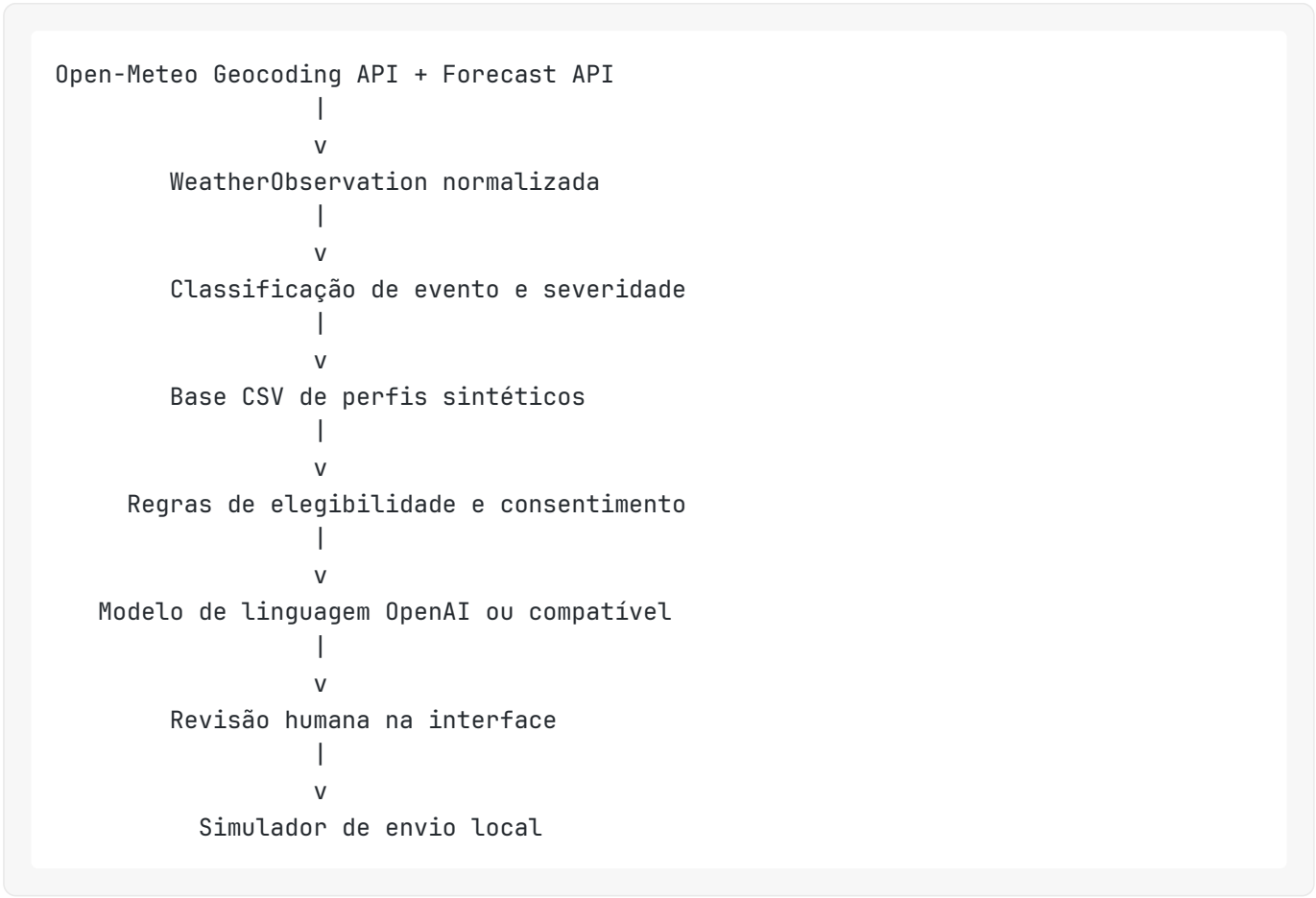
O envio real de SMS, e-mail, WhatsApp ou push não faz parte do MVP. Nenhuma comunicação externa é disparada.

2. Escopo e limites

O sistema utiliza somente perfis sintéticos armazenados em `data/insured_profiles.csv`. A solução não acessa sistemas de seguradoras, não usa dados reais de clientes, não confirma cobertura, não calcula indenização, não aprova sinistros e não solicita credenciais. A simulação cria registros locais com identificador, canal, destinatário sintético, conteúdo e status.

Esse escopo é compatível com o edital, que informa que a simulação é suficiente e que não são esperadas integrações reais com seguradoras ou soluções prontas para produção [1].

3. Arquitetura da solução



Componente	Responsabilidade	Implementação
Coletor Meteorológico	Geocodificar a localidade e consumir dados atuais	<code>seguramente/weather.py</code>
Normalização	Converter o payload externo para o contrato interno	<code>WeatherObservation</code> em <code>seguramente/models.py</code>
Analista de Eventos	Aplicar limiares e produzir evento, severidade e justificativa	<code>classify_event</code> em <code>seguramente/rules.py</code>
Regras de Negócio	Verificar relevância, produto, canal, contato e consentimento	<code>evaluate_eligibility</code> em <code>seguramente/rules.py</code>
Gerador de Mensagens	Criar e validar comunicação preventiva	<code>seguramente/llm.py</code>
Interface e Revisão	Exibir dados, decisões, mensagem e confirmação humana	<code>app.py</code> com Streamlit
Simulador de Envio	Registrar a operação sem acionar canal externo	<code>simulate_message</code> em <code>seguramente/pipeline.py</code>

4. Agentes e responsabilidades

A solução é estruturada em agentes ou componentes especializados, conforme a recomendação pedagógica do desafio [1]. Cada etapa recebe uma entrada, executa uma responsabilidade única e transfere uma saída rastreável.

Agente	Entrada	Processamento	Saída	Limite
Coletor Meteorológico	Localidade informada	Geocodificação e consulta à Open-Meteo	Payload meteorológico	Não decide elegibilidade
Analista de Eventos	<code>WeatherObservation</code>	Comparação com limiares	<code>WeatherEvent</code>	Não escolhe destinatários
Regras de Negócio	Evento e perfis CSV	Verificação de produto, canal, contato e consentimento	Decisões elegível/bloqueado	Não ignora consentimento
Gerador de Mensagens	Evento e perfil elegível	Prompt preventivo para modelo de linguagem	Texto para revisão	Não promete cobertura nem solicita segredo
Simulador de Envio	Mensagem revisada	Registro local da operação	<code>SimulationRecord</code>	Não realiza disparo real

5. Integração meteorológica

A implementação utiliza duas APIs públicas da Open-Meteo: a Geocoding API para converter o nome da localidade em coordenadas e a Forecast API para obter os indicadores atuais. A documentação oficial da fonte está disponível em [Open-Meteo Weather API](#). Nenhuma chave de API é incluída ou necessária para essa consulta pública.

A chamada de previsão solicita os campos `temperature_2m`, `precipitation`, `wind_speed_10m`, `wind_gusts_10m` e `weather_code` em `current`, além de `precipitation_probability` em `hourly`, com timezone UTC. A resposta é convertida para `WeatherObservation`, que preserva fonte, URL, localidade, coordenadas, horário, indicadores e payload bruto para rastreabilidade.

A integração contém tratamento de erro para falha de rede, status HTTP inválido, JSON inválido e localidade não encontrada. O timeout padrão é de 15 segundos.

6. Classificação de eventos

Os limiares do MVP são determinísticos e estão implementados em `seguramente/rules.py`.

Evento	Indicador	Limiar	Severidade	Produtos relacionados
Granizo	<code>weather_code</code> WMO	96 ou 99	Alta	Automóvel, residencial
Alagamento	Precipitação atual	≥ 30 mm	Crítica	Residencial, empresarial
Ventos fortes	Rajada	≥ 60 km/h	Alta	Residencial, empresarial, automóvel
Chuva intensa	Precipitação ou probabilidade	≥ 10 mm ou ≥ 70%	Moderada	Residencial, empresarial
Sem evento relevante	Todos os indicadores	Nenhum limiar atingido	Baixa	Nenhum

Quando mais de uma regra é satisfeita, a ordem de avaliação prioriza granizo, alagamento, ventos fortes e chuva intensa. A justificativa da classificação é armazenada no objeto do evento e exibida na interface.

7. Base sintética e regras de elegibilidade

A base de perfis contém os campos `profile_id`, `name`, `city`, `state`, `product`, `channel`, `consent` e `contact`. A leitura é validada contra esse esquema antes da execução.

Um perfil é elegível somente quando o evento é relevante, o produto possui relação com o evento, o canal pertence ao conjunto suportado (`email`, `sms` ou `push`), o contato correspondente está preenchido e o consentimento está ativo. Quando qualquer requisito falha, o perfil é bloqueado e o motivo é registrado. A interface exibe tanto perfis elegíveis quanto bloqueados.

8. Geração de mensagens com IA

O componente `OpenAICompatibleProvider` utiliza o endpoint `/chat/completions` de uma API OpenAI ou compatível. O provedor, o endpoint e o modelo são configurados por variáveis de ambiente; nenhuma chave é gravada no código ou no repositório. A execução validada utilizou o modelo `gpt-5-mini` do endpoint compatível disponível no ambiente de demonstração.

O prompt de sistema instrui o modelo a produzir uma mensagem curta em português do Brasil, preventiva, clara e não alarmista. O contexto enviado contém somente o evento, a severidade, a justificativa, o nome sintético, o produto, o canal e a localidade do perfil.

A saída passa por `validate_message`, que rejeita mensagem vazia, texto superior a 500 caracteres e padrões relacionados a promessa de cobertura, indenização, aprovação de sinistro, senha, token, credencial ou código de segurança. A mensagem aprovada é apresentada ao usuário antes de qualquer registro de simulação.

Para execução sem credencial, o projeto fornece `TemplateProvider`, explicitamente identificado como modo offline de teste. Esse provider não é o modo de IA; ele permite testar a orquestração e os guardrails sem depender da rede. A demonstração de IA está registrada separadamente em `evidence/demo_run_openai.json`.

9. Fluxo ponta a ponta

1. O usuário informa uma localidade ou seleciona uma fixture demonstrativa.
2. O coletor consulta a Open-Meteo e obtém coordenadas e previsão atual.
3. O sistema normaliza os campos meteorológicos para `WeatherObservation`.
4. O analista compara os indicadores com os limiares e cria um `WeatherEvent`.
5. A base sintética é carregada e cada perfil é avaliado pelas regras.
6. Para perfis elegíveis, o gerador solicita uma mensagem preventiva ao modelo.
7. A interface exibe a mensagem, o evento, o perfil, o canal e as justificativas para revisão humana.
8. Após confirmação, o simulador cria `SimulationRecord` e informa que nenhum canal externo foi acionado.

10. Evidências de funcionamento

A execução offline reproduzível foi realizada com o cenário de chuva intensa e registrou quatro perfis elegíveis, três bloqueados, quatro mensagens e quatro simulações. A execução com modelo de linguagem foi realizada com o cenário de chuva intensa e o provider `openai`, utilizando `gpt-5-mini`; o resultado está em `evidence/demo_run_openai.json`.

Evidência	Arquivo ou comando	Resultado validado
Testes automatizados	<code>pytest -q</code>	7 testes aprovados
Demonstração offline	<code>python scripts/run_demo.py --scenario chuva_intensa --provider template --simulate</code>	Evento, decisões, mensagens e simulações
Demonstração com IA	<code>python scripts/run_demo.py --scenario chuva_intensa --provider openai --simulate</code>	Mensagens geradas pelo modelo e guardrails
Interface	<code>streamlit run app.py</code>	Fluxo visual e revisão humana

Os JSONs de evidência não contêm chaves de API. Os contatos existentes são sintéticos e não representam segurados reais.

11. Tecnologias

Tecnologia	Uso
Python 3.11+	Processamento e orquestração
Streamlit	Interface de demonstração
pandas	Leitura e validação do CSV sintético
requests	Integração com Open-Meteo e API de modelo
Open-Meteo	Fonte pública meteorológica
OpenAI ou API compatível	Geração de mensagens
pytest	Testes de regras, guardrails e simulação
python-dotenv, por configuração	Gestão local de variáveis, quando carregada pelo ambiente

12. Segurança e governança

As credenciais são lidas do ambiente e o `.env` está no `.gitignore`. O contexto do modelo é minimizado para os campos necessários à comunicação. O sistema bloqueia perfis sem consentimento, preserva o motivo das decisões, exige revisão humana e não faz disparos externos.

Essas medidas não transformam o MVP em uma solução de produção. Elas tornam o protótipo demonstrável, rastreável e coerente com o objetivo pedagógico do desafio.

13. Instalação e execução

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# preencher OPENAI_API_KEY no .env
pytest -q
streamlit run app.py
```

Para executar a integração com IA pela linha de comando:

```
set -a && source .env && set +a
python scripts/run_demo.py --scenario chuva_intensa --provider openai --simulate --out
```

14. Rastreabilidade do edital

Item exigido	Evidência no pacote
Fonte meteorológica pública	<code>seguramente/weather.py</code> , documentação da Open-Meteo e evidência JSON
Identificação automática	<code>classify_event</code> , tabela de limiares e testes
Regras de decisão	<code>evaluate_eligibility</code> , CSV sintético, motivos exibidos e testes
Mensagens com IA	<code>OpenAICompatibleProvider</code> , prompt, guardrails e evidência OpenAI
Simulação de envio	<code>simulate_message</code> , interface e JSON de evidência
Fluxo completo	<code>app.py</code> , <code>run_demo.py</code> , testes e relatório
Arquitetura e agentes	Seções 3 e 4 deste relatório
Tecnologias e fluxo	Seções 5, 9 e 11 deste relatório
Exemplos de mensagens	<code>evidence/demo_run_openai.json</code> e interface
Código e instalação	Repositório, <code>README.md</code> , <code>requirements.txt</code> e <code>LICENSE</code>

15. Limitações conhecidas

O MVP analisa a observação atual retornada pela fonte pública e não mantém histórico de eventos. A classificação usa limiares didáticos, não substitui alerta oficial nem avaliação técnica de risco. O modo offline é destinado a testes; a demonstração de IA requer credencial e modelo disponíveis. O simulador não verifica entrega em canais reais.

Referências

[1]: `Desafios(1).pdf` , Instituto de Inteligência Artificial Aplicada — I2A2, páginas 16–19, documento do edital fornecido para a entrega.